

fSIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

CO4 ASSESSMENT TOOL 2

CI/CD Pipeline Design Exercise

Intelligent Disaster Early Warning and Evacuation Management System

Course Code	CSA10
Course Title	Software Engineering
CO Assessed	CO4
Assessment	Assessment Tool 2 – CI/CD Pipeline Design Exercise
CO4 Statement	Implement robust testing, quality assurance, and DevOps practices, emphasizing security, reliability, and ethics
Weightage	20%
Total Marks	25
Duration	60 minutes

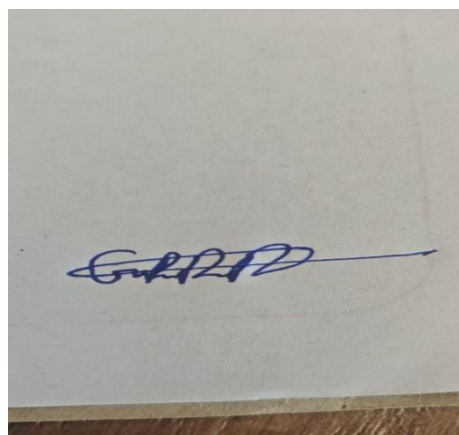
Student Name: G. Pradeep Ranga Reddy

Reg. No.: 192311289

Date: 19-08-2026

Code of Conduct

I, G. Pradeep Ranga Reddy 192311289(Name / Reg. No.), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.



Signature of the Student:

Problem Statement Overview

Intelligent Disaster Early Warning and Evacuation Management System

A software system that ingests real-time data from seismic, weather, and IoT sensor networks, applies predictive models to detect disaster risk (floods, earthquakes, cyclones, fires), and issues automated multi-channel alerts while guiding citizens through the nearest safe evacuation routes and shelter allocation. The system is life-critical, must operate under high load during emergencies, integrates with third-party government and telecom APIs, and demands near-zero downtime, strong data integrity, and rapid, safe release cycles — making a disciplined CI/CD pipeline essential.

1. Requirement & Pipeline Analysis

1.1 Nature of the System

The Intelligent Disaster Early Warning and Evacuation Management System (IDEWEMS) is a distributed, mission-critical platform composed of a sensor-data ingestion service, an AI/ML-based risk-prediction engine, a geospatial evacuation-routing module, a multi-channel alert dispatcher (SMS, push notification, sirens, IVR), and a citizen-facing mobile/web application. Because the system directly affects public safety, every code change must be validated rigorously before it reaches production, and releases must be deployable and reversible within minutes.

1.2 Identified CI/CD Requirements

- **High reliability and zero-downtime releases:** outages during an active disaster event are unacceptable, so the pipeline must support blue-green or canary deployments and instant rollback.
- **Rapid, frequent integration:** multiple teams (sensor integration, ML, mapping, alerting, mobile) commit concurrently; automated build and test on every commit is required to catch integration issues early.
- **Rigorous automated testing:** unit, integration, API, load/stress (to simulate sudden traffic spikes during a disaster), and geospatial-accuracy tests must run automatically before any promotion.
- **Security and data integrity:** the system handles location data, personal contact details, and government feeds, so static/dynamic security scanning, dependency scanning, and secrets management are mandatory.
- **Scalable, environment-parity deployment:** Dev, Staging, and Production environments must mirror each other (containerized) so that performance under simulated disaster load is verified before go-live.
- **Auditability and compliance:** every build, test result, and deployment must be logged and traceable, since the system may be reviewed by disaster-management authorities.
- **Fast feedback and observability:** the DevOps team needs immediate notification of failures and real-time monitoring once a version is live, given the life-safety impact of defects.

1.3 Key Non-Functional Drivers for Pipeline Design

Driver	Impact on CI/CD Design
Availability	Blue-green/canary deployment, automated health checks, instant rollback
Scalability	Load testing in staging with disaster-scale simulated traffic before promotion
Data Accuracy	Automated validation of prediction-model outputs and routing/geospatial data
Security & Privacy	SAST/DAST, dependency & container scanning, secrets vault, RBAC on pipeline

Driver	Impact on CI/CD Design
Traceability	Versioned artifacts, immutable build logs, deployment audit trail

2. Pipeline Architecture & Workflow

The pipeline is organized into five logical zones — Source & Trigger, Continuous Integration, Packaging, Continuous Deployment, and Operate & Observe — executed sequentially for every change, with automated feedback loops (notifications and rollback) running in parallel. The end-to-end flow from a developer commit through to a monitored production release is shown below.

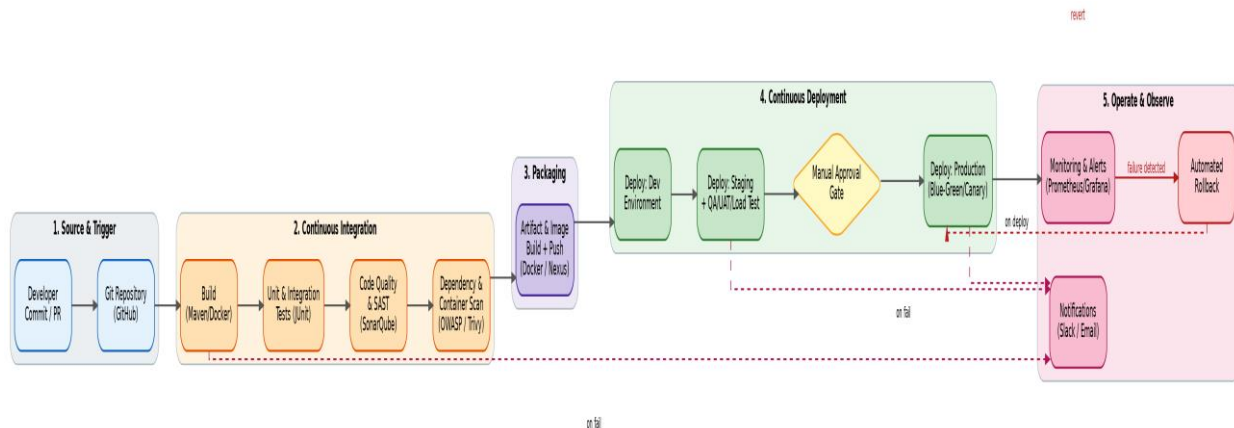


Figure 1: CI/CD Pipeline Architecture – Intelligent Disaster Early Warning and Evacuation Management System

2.1 Pipeline Stages

1. **Source Code Management:** Developers commit feature branches (e.g. sensor-ingestion, alert-dispatcher, routing-engine) to Git; a pull request into the main branch automatically triggers the pipeline via a webhook.
2. **Build:** The application and its containerized microservices are compiled/built (e.g. Maven for backend services, npm build for the citizen web app) and dependencies are resolved.
3. **Automated Testing:** Unit tests and integration tests run against the build to validate business logic — including risk-prediction accuracy checks and route-calculation correctness.
4. **Code Quality & Security Analysis:** Static code analysis and static application security testing (SAST) evaluate code smells, complexity, and vulnerabilities; a quality gate blocks the pipeline if thresholds are not met.
5. **Dependency & Container Scanning:** Open-source dependencies and the built Docker image are scanned for known CVEs before packaging is allowed to proceed.
6. **Packaging:** A versioned, immutable Docker image is built and pushed to a container/artifact registry, tagged with the commit SHA and semantic version.
7. **Deploy to Development:** The image is automatically deployed to the Dev environment for developer smoke-testing and early integration checks.
8. **Deploy to Staging & Extended Testing:** The build is promoted to a production-like Staging environment, where QA, UAT, and disaster-scale load/stress testing run automatically.
9. **Manual Approval Gate:** A release manager (or disaster-management domain reviewer) reviews staging results and approves promotion to Production — a deliberate control point given the safety-critical nature of the system.
10. **Deploy to Production:** A blue-green or canary deployment strategy shifts live traffic to the new version gradually, minimizing risk to an active alerting system.

11. Monitor & Observe: Real-time monitoring tracks latency, alert-delivery success rate, and error rates; anomalies trigger automated alerts.
12. Rollback (on failure): If monitoring or health checks detect degraded alert-delivery or prediction accuracy, the pipeline automatically reverts to the last stable version.

3. Tools & Technology Selection

Tools were selected for strong community support, native containerization compatibility, and suitability for a life-critical, high-availability system.

Stage	Tool(s)	Justification
Source Code Mgmt.	Git + GitHub	Industry-standard distributed VCS; branch protection rules and PR-based review enforce integration discipline for a multi-team, safety-critical codebase.
CI Orchestration	Jenkins / GitHub Actions	Widely adopted, highly extensible pipeline-as-code (Jenkinsfile / YAML workflows); large plugin ecosystem supports every downstream stage.
Build	Maven (services) / npm (web & mobile)	Standard, reproducible build tooling for Java-based microservices and JavaScript front-end/mobile clients respectively.
Unit & Integration Testing	JUnit, Mockito, Postman/Newman	JUnit/Mockito validate business logic (prediction and routing modules) in isolation; Postman/Newman automates REST API contract testing between microservices.
Code Quality	SonarQube	Continuous static analysis with configurable quality gates; blocks merges that introduce high complexity, duplication, or bugs.
Security Scanning	OWASP Dependency-Check, Trivy, SAST (Semgrep)	Detects vulnerable dependencies, container image CVEs, and insecure code patterns before deployment — critical given exposure to public/government data feeds.
Packaging	Docker + Nexus/Harbor Registry	Containerization guarantees environment parity across Dev/Staging/Prod; a private registry provides versioned, access-controlled artifact storage.
Deployment	Kubernetes + Helm	Enables declarative, repeatable deployments and native support for blue-green/canary rollout strategies with automated health checks.
Secrets Management	HashiCorp Vault	Centralizes API keys/credentials for third-party telecom and government integrations instead of embedding them in code or pipeline scripts.
Monitoring	Prometheus + Grafana, ELK Stack	Real-time metrics (alert-delivery latency, prediction throughput) and centralized log aggregation for rapid incident diagnosis.
Notifications	Slack / Email (via CI webhooks)	Immediate build/deployment status alerts to the DevOps and disaster-response teams, minimizing mean-time-to-awareness.

4. Automated Testing & Quality Integration

Given the life-safety implications of a defect (a missed evacuation alert or an incorrect safe route), testing is layered so that each pipeline stage catches a different class of failure before promotion.

Testing Layer	Focus	Pipeline Stage
Unit Testing	Individual functions – risk-scoring logic, route-distance calculations, alert-message formatting	CI – immediately after build
Integration Testing	Interaction between sensor-ingestion, prediction engine, routing, and alert-dispatch services	CI – after unit tests pass
API / Contract Testing	REST/webhook contracts with telecom SMS gateways and government sensor feeds	CI – automated via Postman/Newman
Static Code Analysis (SAST)	Code smells, complexity, insecure coding patterns	Quality gate before packaging
Security Testing (DAST/Dependency)	Vulnerable libraries, container CVEs, exposed secrets	Before artifact promotion
Load & Stress Testing	Alert-dispatch throughput and system stability under a simulated mass-disaster traffic spike	Staging environment
Geospatial Accuracy Testing	Validation of evacuation-route correctness against known map/shelter datasets	Staging environment
User Acceptance Testing (UAT)	End-to-end citizen and control-room operator workflows	Staging – before approval gate
Smoke / Health-Check Testing	Post-deployment sanity checks on critical endpoints	Immediately after Dev and Prod deploy

A quality gate is enforced in SonarQube (e.g. minimum 80% code coverage, zero critical/blocker issues, no critical CVEs) — any failure at this gate automatically halts the pipeline and prevents packaging, ensuring that only verified, secure builds are eligible for deployment.

5. Deployment, Security & Rollback Strategy

5.1 Deployment Environments

Environment	Purpose	Deployment Trigger
Development	Continuous validation for developers; latest build always available for smoke testing	Automatic, on every successful merge to main
Staging / Test	Production-like environment for QA, UAT, load, and geospatial-accuracy testing	Automatic, after Dev smoke tests pass
Production	Live system serving citizens, control-room operators, and partner agencies	Manual approval gate, then blue-green/canary automated rollout

5.2 Deployment Strategy

- **Blue-Green Deployment:** a fully provisioned parallel (“green”) environment is deployed alongside the live (“blue”) one; traffic is switched only after automated health checks pass, allowing near-instant rollback by re-pointing traffic.

- **Canary Releases:** for high-risk changes (e.g. updates to the prediction model or alert-dispatch logic), a small percentage of traffic is routed to the new version first, with automatic promotion or rollback based on live error-rate and latency thresholds.
- **Feature Flags:** new, higher-risk capabilities (e.g. a new sensor-data source) are shipped behind flags so they can be disabled instantly without a full redeployment.

5.3 Security Practices

- **Secrets management:** all API keys and credentials for telecom/government integrations are stored in HashiCorp Vault and injected at runtime, never hard-coded or committed.
- **Static & dynamic scanning:** SAST, dependency scanning, and container image scanning run at every build; the pipeline fails closed on critical findings.
- **Role-based access control (RBAC):** pipeline and production deployment permissions are restricted to authorized release managers; production deploys require the manual approval gate.
- **Immutable, signed artifacts:** container images are versioned and cryptographically signed, ensuring only verified builds can be deployed.
- **Audit logging:** every pipeline run, approval, and deployment is logged for compliance review by disaster-management authorities.

5.4 Monitoring, Notifications & Rollback

- **Real-time monitoring:** Prometheus and Grafana track alert-delivery latency, prediction-service throughput, and error rates; the ELK stack aggregates logs for root-cause analysis.
- **Automated notifications:** build failures, quality-gate failures, and deployment outcomes are pushed to Slack/email so the DevOps and disaster-response teams are informed within seconds.
- **Automated rollback:** if post-deployment health checks or monitoring thresholds are breached (e.g. alert-delivery success rate drops below a defined SLA), the pipeline automatically triggers a rollback to the last known-stable version via Kubernetes/Helm rollback, minimizing downtime for a life-critical system.
- **Database migration safety:** schema changes use backward-compatible, versioned migration scripts with a documented rollback path, so a code rollback never leaves the database in an incompatible state.

6. Workflow Explanation: Commit to Deployment

The following walkthrough traces a single code change — for example, an improvement to the flood-risk prediction model — through the entire pipeline shown in Figure 1:

13. A developer pushes a commit to a feature branch and opens a pull request; this automatically triggers the CI pipeline via a GitHub webhook.
14. The pipeline builds the affected microservice and executes unit and integration tests; any failure immediately stops the pipeline and notifies the developer on Slack.
15. SonarQube performs static analysis and enforces the quality gate; OWASP Dependency-Check and Trivy scan dependencies and the container image for vulnerabilities.
16. On success, a versioned Docker image is built and pushed to the artifact registry, tagged with the commit SHA.
17. The image is auto-deployed to the Development environment for smoke testing, then promoted to Staging, where QA, UAT, load testing, and geospatial-accuracy tests run automatically.
18. A release manager reviews the staging test report at the manual approval gate and authorizes production release.
19. The pipeline performs a canary (or blue-green) rollout to Production, gradually shifting live traffic while automated health checks and Prometheus/Grafana metrics are monitored.

20. If all health checks pass, the rollout completes and a deployment-success notification is sent; if any threshold is breached, the pipeline automatically rolls back to the previous stable version and alerts the on-call team.

7. Conclusion

For the Intelligent Disaster Early Warning and Evacuation Management System, the designed CI/CD pipeline embeds automated testing, static/dynamic security scanning, and a manual approval gate at every stage between commit and production, while blue-green/canary deployment and automated rollback ensure that the life-critical alerting and evacuation-guidance services remain available and trustworthy at all times. This combination of speed (frequent, automated integration) and discipline (quality gates, security scanning, staged promotion, observability) directly supports CO4 — robust testing, quality assurance, and DevOps practice with an explicit emphasis on security, reliability, and ethics in a public-safety context.